Comparison of covariance estimators for PoSIRd

Daniela

2026-08-26

Contents

1 Introduction **2**

2 Simulation parameters **2**

3 Generate changepoints and signal **3**

4 Segmentation scenarios **6**
4.1 Correct segmentation 6
4.2 Undersegmentation 6
4.3 Oversegmentation 6

5 Helper functions **7**
5.1 Segment means and lengths 7
5.2 Raw first-difference estimators 8
5.3 Covariance diagnostics 9

6 One simulation **10**

7 Simulation study **14**

8 Estimation error **18**
8.1 Relative Frobenius error 18
8.2 Relative operator-norm error 20

9 Positive definiteness **21**

10 Condition number **22**

11 PoSIRd coverage and confidence-region area **23**

12 True signal and targets **23**
12.1 True signal at every observation 23
12.2 True target associated with a supplied segmentation 24

13 Construct PoSIRd regions with a chosen covariance estimator **25**

14 Evaluate coverage and ellipse area **26**

15 One PoSIRd example **29**

16 Monte Carlo evaluation of PoSIRd **32**

17 Simultaneous coverage **37**

18 Confidence-region areas **38**
18.1 Numerical summary 38

19 Mean ellipse area **40**

20 Median ellipse area **41**

21 Relative area with respect to the correct pooled estimator **42**

22 Coverage and area together **43**

23 Interpretation **44**

1 Introduction

The aim of this vignette is to compare the covariance estimators available for PoSIRd.

We consider three estimators:

- pooled covariance based on residuals from the estimated segmentation:

$$\hat{\Sigma}_{\text{pool}} = \frac{1}{n-K} \sum_{k=1}^{K} \sum_{i \in I_k} (X_i - \hat{\mu}_k)(X_i - \hat{\mu}_k)^\top;$$

- first-difference estimator with entrywise median aggregation:

$$\hat{\Sigma}_{\text{EW}} = \left[\text{median}_{b=1,\dots,B} \left\{ (\hat{\Sigma}_b)_{j\ell} \right\} \right]_{j,\ell=1}^{d}, \qquad \hat{\Sigma}_b = \frac{1}{2|I_b|} \sum_{i \in I_b} \Delta_i \Delta_i^\top, \quad \Delta_i = X_{i+1} - X_i;$$

- first-difference estimator with geometric-median aggregation:

$$\hat{\Sigma}_{\text{GM}} = \arg\min_{M \in \mathbb{R}^{d \times d}} \sum_{b=1}^{B} \|\hat{\Sigma}_b - M\|_F,$$

  where $\hat{\Sigma}_b$ is defined as above. The pooled estimator uses the supplied segmentation and is therefore expected to perform well when the segmentation is correct, but may be affected by missed changepoints.

The first-difference estimators do not use the segmentation. They estimate the covariance from consecutive differences and use robust aggregation to reduce the effect of differences crossing true changepoints.

We investigate:

- estimation error with respect to the true covariance matrix;
- eigenvalues and condition number of the estimators;
- frequency with which the raw MoM estimators are not positive definite;
- sensitivity to correct segmentation, undersegmentation and oversegmentation.

```
library(mvtnorm)
```

2 Simulation parameters

We consider a two-dimensional Gaussian piecewise-constant signal

$$X_i = \mu_i + \varepsilon_i, \qquad \varepsilon_i \sim N(0, \Sigma),$$

with

$$\Sigma = \begin{pmatrix} 1 & \rho \\ \rho & 1 \end{pmatrix}.$$

```
Qtable_path <- here::here(
  "Simulations",
  "Table_quantiles.txt"
)

message("Quantile path: ", Qtable_path)
```

```
## Quantile path: /home/dcorbett/github/posir/Simulations/Table_quantiles.txt
```

```
if (!file.exists(Qtable_path)) {
  stop(
    "Quantile table not found at: ",
    Qtable_path
  )
}

Qtable <- read_quantiles(
  Qtable_path
)
```

```
n <- 1000
n_chg <- 10
n_segments <- n_chg + 1

delta <- 0.05
alpha <- 0.05

rho <- 0.5
Sigma <- matrix(
  c(
    1, rho,
    rho, 1
  ),
  nrow = 2
)

possible_states <- c(
  -2,
  0,
  2
)

n_sim <- 1000

covariance_blocks <- 10L
```

3 Generate changepoints and signal

We generate changepoints under the constraint that every true segment has length at least $n\delta$.

```
generate_changepoints <- function(
    n,
    n_chg,
    delta
) {
  n_segments <- n_chg + 1L
  min_length <- ceiling(delta * n)

  remaining_length <-
    n - n_segments * min_length

  if (remaining_length < 0L) {
    stop(
      "The requested number of changepoints is incompatible with delta."
    )
  }

  segment_lengths <- min_length +
    as.vector(
      rmultinom(
        n = 1,
        size = remaining_length,
        prob = rep(1, n_segments)
      )
    )

  changepoints <- cumsum(
    segment_lengths
  )[-n_segments]

  list(
    changepoints = changepoints,
    lengths = segment_lengths
  )
}
```

Generate segment means, requiring the mean to change in both coordinates at every changepoint.

```
generate_means <- function(
    n_segments,
    possible_states
) {
  means <- matrix(
    NA_real_,
    nrow = n_segments,
    ncol = 2
  )

  means[1, ] <- sample(
    possible_states,
    size = 2,
    replace = TRUE
  )
```

```
  for (i in 2:n_segments) {
    means[i, 1] <- sample(
      setdiff(
        possible_states,
        means[i - 1, 1]
      ),
      size = 1
    )

    means[i, 2] <- sample(
      setdiff(
        possible_states,
        means[i - 1, 2]
      ),
      size = 1
    )
  }

  colnames(means) <- c(
    "mu1",
    "mu2"
  )

  means
}
```

Generate observations.

```
generate_data <- function(
    n,
    changepoints,
    means,
    Sigma
) {
  boundaries <- c(
    0,
    changepoints,
    n
  )

  x <- matrix(
    NA_real_,
    nrow = n,
    ncol = 2
  )

  for (i in seq_len(nrow(means))) {
    current_rows <-
      (boundaries[i] + 1L):boundaries[i + 1L]

    x[current_rows, ] <- rmvnorm(
      n = length(current_rows),
      mean = means[i, ],
      sigma = Sigma
```

```
    )
  }

  colnames(x) <- c(
    "Y1",
    "Y2"
  )

  x
}
```

4 Segmentation scenarios

We consider three situations.

4.1 Correct segmentation

The pooled estimator receives the true changepoints.

4.2 Undersegmentation

We remove every second true changepoint. Therefore, some estimated segments contain multiple true means.

```
undersegment <- function(
    true_changepoints
) {
  true_changepoints[
    seq(
      2L,
      length(true_changepoints),
      by = 2L
    )
  ]
}
```

4.3 Oversegmentation

We add a changepoint at the midpoint of every true segment whenever possible.

```
oversegment <- function(
    true_changepoints,
    n
) {
  boundaries <- c(
    0,
    true_changepoints,
    n
  )

  additional_changepoints <- vapply(
    seq_len(length(boundaries) - 1L),
    function(i) {
      as.integer(
        floor(
```

```
          (
            boundaries[i] +
            boundaries[i + 1L]
          ) / 2
        )
      )
    },
    integer(1)
  )

  sort(
    unique(
      c(
        true_changepoints,
        additional_changepoints
      )
    )
  )
}
```

5 Helper functions

5.1 Segment means and lengths

```
get_segment_information <- function(
    x,
    breakpoints
) {
  n <- nrow(x)

  boundaries <- c(
    0,
    breakpoints,
    n
  )

  lengths <- diff(boundaries)

  means <- t(
    vapply(
      seq_len(length(lengths)),
      function(i) {
        current_rows <-
          (boundaries[i] + 1L):boundaries[i + 1L]

        colMeans(
          x[
            current_rows,
            ,
            drop = FALSE
          ]
        )
```

```
      },
      numeric(ncol(x))
    )
  )

  list(
    means = means,
    lengths = lengths
  )
}
```

5.2 Raw first-difference estimators

For this simulation we need the covariance estimates *before* the positive-definite correction, because we want to record how often each aggregation method naturally produces a non-PD matrix.

```
estimate_covariance_diff_mom_raw <- function(
    x,
    number_of_blocks,
    aggregation = c(
      "entrywise",
      "geometric"
    )
) {
  aggregation <- match.arg(
    aggregation
  )

  n <- nrow(x)

  first_differences <-
    x[-1L, , drop = FALSE] -
    x[-n, , drop = FALSE]

  block_indices <- split_into_consecutive_blocks(
    number_of_observations = n - 1L,
    number_of_blocks = number_of_blocks
  )

  block_covariances <- lapply(
    block_indices,
    function(indices) {
      current_differences <-
        first_differences[
          indices,
          ,
          drop = FALSE
        ]

      crossprod(
        current_differences
      ) / (
        2 * length(indices)
      )
```

```
    }
  )

  switch(
    aggregation,

    entrywise = {
      entrywise_matrix_median(
        block_covariances
      )
    },

    geometric = {
      geometric_matrix_median(
        block_covariances
      )
    }
  )
}
```

5.3 Covariance diagnostics

```
covariance_diagnostics <- function(
    covariance,
    true_covariance
) {
  eigenvalues <- eigen(
    covariance,
    symmetric = TRUE,
    only.values = TRUE
  )$values

  difference <-
    covariance - true_covariance

  frobenius_error <- sqrt(
    sum(difference^2)
  )

  relative_frobenius_error <-
    frobenius_error /
    sqrt(sum(true_covariance^2))

  operator_error <- max(
    abs(
      eigen(
        difference,
        symmetric = TRUE,
        only.values = TRUE
      )$values
    )
  )
```

```
  true_operator_norm <- max(
    abs(
      eigen(
        true_covariance,
        symmetric = TRUE,
        only.values = TRUE
      )$values
    )
  )

  relative_operator_error <-
    operator_error /
    true_operator_norm

  list(
    relative_frobenius_error =
      relative_frobenius_error,
    relative_operator_error =
      relative_operator_error,
    minimum_eigenvalue =
      min(eigenvalues),
    maximum_eigenvalue =
      max(eigenvalues),
    condition_number =
      if (min(eigenvalues) > 0) {
        max(eigenvalues) /
          min(eigenvalues)
      } else {
        Inf
      },
    positive_definite =
      all(eigenvalues > 0)
  )
}
```

6 One simulation

Before running the full experiment, inspect the three estimators in one simulation.

```
set.seed(1418)

segmentation <- generate_changepoints(
  n = n,
  n_chg = n_chg,
  delta = delta
)

true_chg <- segmentation$changepoints

true_means <- generate_means(
  n_segments = n_segments,
  possible_states = possible_states
)
```

```
x <- generate_data(
  n = n,
  changepoints = true_chg,
  means = true_means,
  Sigma = Sigma
)

under_chg <- undersegment(
  true_chg
)

over_chg <- oversegment(
  true_changepoints = true_chg,
  n = n
)
```

Compute the pooled estimator under the three segmentations.

```
pooled_estimates <- lapply(
  list(
    correct = true_chg,
    under = under_chg,
    over = over_chg
  ),
  function(breakpoints) {
    segment_information <-
      get_segment_information(
        x = x,
        breakpoints = breakpoints
      )

    estimate_covariance_pooled(
      x = x,
      means = segment_information$means,
      lengths = segment_information$lengths
    )
  }
)

pooled_estimates
```

```
## $correct
##           Y1        Y2
## Y1 0.9253390 0.4657728
## Y2 0.4657728 0.9678733
##
## $under
##          Y1       Y2
## Y1 3.099056 2.142831
## Y2 2.142831 3.153493
##
## $over
##           Y1        Y2
## Y1 0.9221785 0.4662772
```

```
## Y2 0.4662772 0.9696557
```

The two first-difference estimators do not depend on the supplied segmentation.

```
cov_entrywise_raw <-
  estimate_covariance_diff_mom_raw(
    x = x,
    number_of_blocks = covariance_blocks,
    aggregation = "entrywise"
  )

cov_geometric_raw <-
  estimate_covariance_diff_mom_raw(
    x = x,
    number_of_blocks = covariance_blocks,
    aggregation = "geometric"
  )

cov_entrywise_raw
```

```
##           Y1        Y2
## Y1 0.9277732 0.4828794
## Y2 0.4828794 1.0873765
```

```
cov_geometric_raw
```

```
##           Y1        Y2
## Y1 0.9408494 0.5019343
## Y2 0.5019343 1.0293685
```

Compare them with the true covariance.

```
Sigma
```

```
##      [,1] [,2]
## [1,]  1.0  0.5
## [2,]  0.5  1.0
```

```
covariance_diagnostics(
  pooled_estimates$correct,
  Sigma
)
```

```
## $relative_frobenius_error
## [1] 0.05983109
##
## $relative_operator_error
## [1] 0.06246013
##
## $minimum_eigenvalue
## [1] 0.480348
##
## $maximum_eigenvalue
## [1] 1.412864
##
## $condition_number
## [1] 2.941334
##
```

```
## $positive_definite
## [1] TRUE
```

```
covariance_diagnostics(
  pooled_estimates$under,
  Sigma
)
```

```
## $relative_frobenius_error
## [1] 2.403444
##
## $relative_operator_error
## [1] 2.512887
##
## $minimum_eigenvalue
## [1] 0.9832703
##
## $maximum_eigenvalue
## [1] 5.269278
##
## $condition_number
## [1] 5.358931
##
## $positive_definite
## [1] TRUE
```

```
covariance_diagnostics(
  pooled_estimates$over,
  Sigma
)
```

```
## $relative_frobenius_error
## [1] 0.06083228
##
## $relative_operator_error
## [1] 0.06354869
##
## $minimum_eigenvalue
## [1] 0.479036
##
## $maximum_eigenvalue
## [1] 1.412798
##
## $condition_number
## [1] 2.949252
##
## $positive_definite
## [1] TRUE
```

```
covariance_diagnostics(
  cov_entrywise_raw,
  Sigma
)
```

```
## $relative_frobenius_error
## [1] 0.07331464
```

```
##
## $relative_operator_error
## [1] 0.05946157
##
## $minimum_eigenvalue
## [1] 0.5181458
##
## $maximum_eigenvalue
## [1] 1.497004
##
## $condition_number
## [1] 2.889156
##
## $positive_definite
## [1] TRUE
```

```
covariance_diagnostics(
  cov_geometric_raw,
  Sigma
)
```

```
## $relative_frobenius_error
## [1] 0.04180329
##
## $relative_operator_error
## [1] 0.03946193
##
## $minimum_eigenvalue
## [1] 0.4812271
##
## $maximum_eigenvalue
## [1] 1.488991
##
## $condition_number
## [1] 3.094154
##
## $positive_definite
## [1] TRUE
```

7 Simulation study

We now repeat the experiment and record the estimation error and spectral properties of each estimator.

```
set.seed(1442)

segmentation_methods <- c(
  "correct",
  "under",
  "over"
)

covariance_methods <- c(
  "pooled",
  "diff_mom_entrywise",
  "diff_mom_geometric"
```

```
)

results <- vector(
  "list",
  n_sim * (
    length(segmentation_methods) +
    2L
  )
)

result_index <- 1L

for (sim in seq_len(n_sim)) {

  # ------------------------------------------------------------
  # Generate signal
  # ------------------------------------------------------------

  segmentation <- generate_changepoints(
    n = n,
    n_chg = n_chg,
    delta = delta
  )

  true_chg <- segmentation$changepoints

  true_means <- generate_means(
    n_segments = n_chg + 1L,
    possible_states = possible_states
  )

  x <- generate_data(
    n = n,
    changepoints = true_chg,
    means = true_means,
    Sigma = Sigma
  )

  segmentations <- list(
    correct = true_chg,
    under = undersegment(
      true_chg
    ),
    over = oversegment(
      true_changepoints = true_chg,
      n = n
    )
  )

  # ------------------------------------------------------------
  # Pooled estimator
  # ------------------------------------------------------------
```

```r
for (
  segmentation_name in names(segmentations)
) {
  current_chg <-
    segmentations[[segmentation_name]]

  segment_information <-
    get_segment_information(
      x = x,
      breakpoints = current_chg
    )

  covariance_raw <-
    estimate_covariance_pooled(
      x = x,
      means = segment_information$means,
      lengths = segment_information$lengths
    )

  diagnostics <-
    covariance_diagnostics(
      covariance = covariance_raw,
      true_covariance = Sigma
    )

  results[[result_index]] <- data.frame(
    simulation = sim,
    covariance_method = "pooled",
    segmentation = segmentation_name,
    relative_frobenius_error =
      diagnostics$relative_frobenius_error,
    relative_operator_error =
      diagnostics$relative_operator_error,
    minimum_eigenvalue =
      diagnostics$minimum_eigenvalue,
    maximum_eigenvalue =
      diagnostics$maximum_eigenvalue,
    condition_number =
      diagnostics$condition_number,
    raw_positive_definite =
      diagnostics$positive_definite
  )

  result_index <- result_index + 1L
}

# ------------------------------------------------------------
# First-difference estimators
# ------------------------------------------------------------

for (
  aggregation in c(
    "entrywise",
```

```
      "geometric"
    )
  ) {
    covariance_raw <-
      estimate_covariance_diff_mom_raw(
        x = x,
        number_of_blocks = covariance_blocks,
        aggregation = aggregation
      )

    diagnostics <-
      covariance_diagnostics(
        covariance = covariance_raw,
        true_covariance = Sigma
      )

    results[[result_index]] <- data.frame(
      simulation = sim,
      covariance_method = paste0(
        "diff_mom_",
        aggregation
      ),
      segmentation = "independent",
      relative_frobenius_error =
        diagnostics$relative_frobenius_error,
      relative_operator_error =
        diagnostics$relative_operator_error,
      minimum_eigenvalue =
        diagnostics$minimum_eigenvalue,
      maximum_eigenvalue =
        diagnostics$maximum_eigenvalue,
      condition_number =
        diagnostics$condition_number,
      raw_positive_definite =
        diagnostics$positive_definite
    )

    result_index <- result_index + 1L
  }

  if (sim %% 100L == 0L) {
    message(
      "Simulation ",
      sim,
      "/",
      n_sim
    )
  }
}
```

```
## Simulation 100/1000

## Simulation 200/1000

## Simulation 300/1000
```

```
## Simulation 400/1000
## Simulation 500/1000
## Simulation 600/1000
## Simulation 700/1000
## Simulation 800/1000
## Simulation 900/1000
## Simulation 1000/1000
```

```
simulation_results <- do.call(
  rbind,
  results
)

rownames(simulation_results) <- NULL
```

8 Estimation error

8.1 Relative Frobenius error

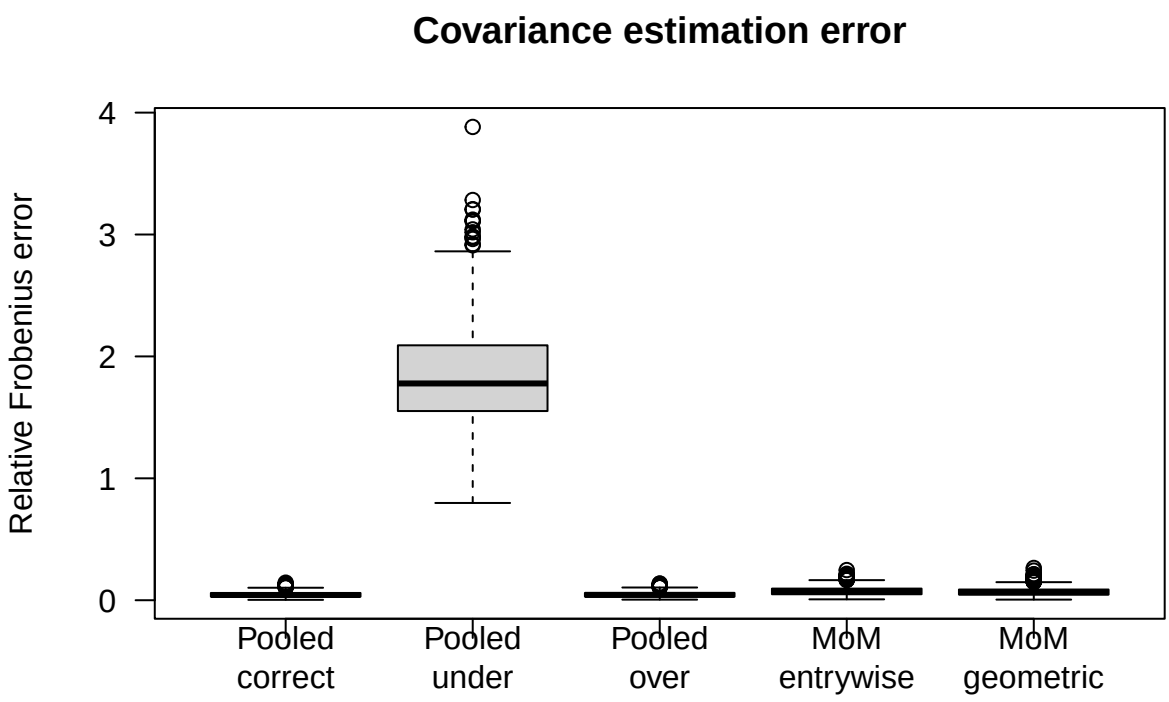
```
aggregate(
  relative_frobenius_error ~
    covariance_method + segmentation,
  data = simulation_results,
  FUN = function(x) {
    c(
      mean = mean(x),
      median = median(x),
      q90 = quantile(x, 0.90)
    )
  }
)
```

```
##    covariance_method segmentation relative_frobenius_error.mean
## 1             pooled      correct                     0.04580976
## 2 diff_mom_entrywise  independent                     0.07411193
## 3 diff_mom_geometric  independent                     0.06999159
## 4             pooled         over                     0.04601939
## 5             pooled        under                     1.83398346
##   relative_frobenius_error.median relative_frobenius_error.q90.90%
## 1                      0.04288340                         0.07466613
## 2                      0.06846213                         0.11885284
## 3                      0.06424791                         0.11213878
## 4                      0.04332686                         0.07552207
## 5                      1.77835732                         2.39999169
```

```
simulation_results$plot_group <- with(
  simulation_results,
  ifelse(
    covariance_method == "pooled",
    paste0("Pooled.", segmentation),
    covariance_method
```

```
  )
)

simulation_results$plot_group <- factor(
  simulation_results$plot_group,
  levels = c(
    "Pooled.correct",
    "Pooled.under",
    "Pooled.over",
    "diff_mom_entrywise",
    "diff_mom_geometric"
  ),
  labels = c(
    "Pooled\ncorrect",
    "Pooled\nunder",
    "Pooled\nover",
    "MoM\nentrywise",
    "MoM\ngeometric"
  )
)
boxplot(
  relative_frobenius_error ~ plot_group,
  data = simulation_results,
  las = 1,
  ylab = "Relative Frobenius error",
  xlab = "",
  main = "Covariance estimation error"
)
```

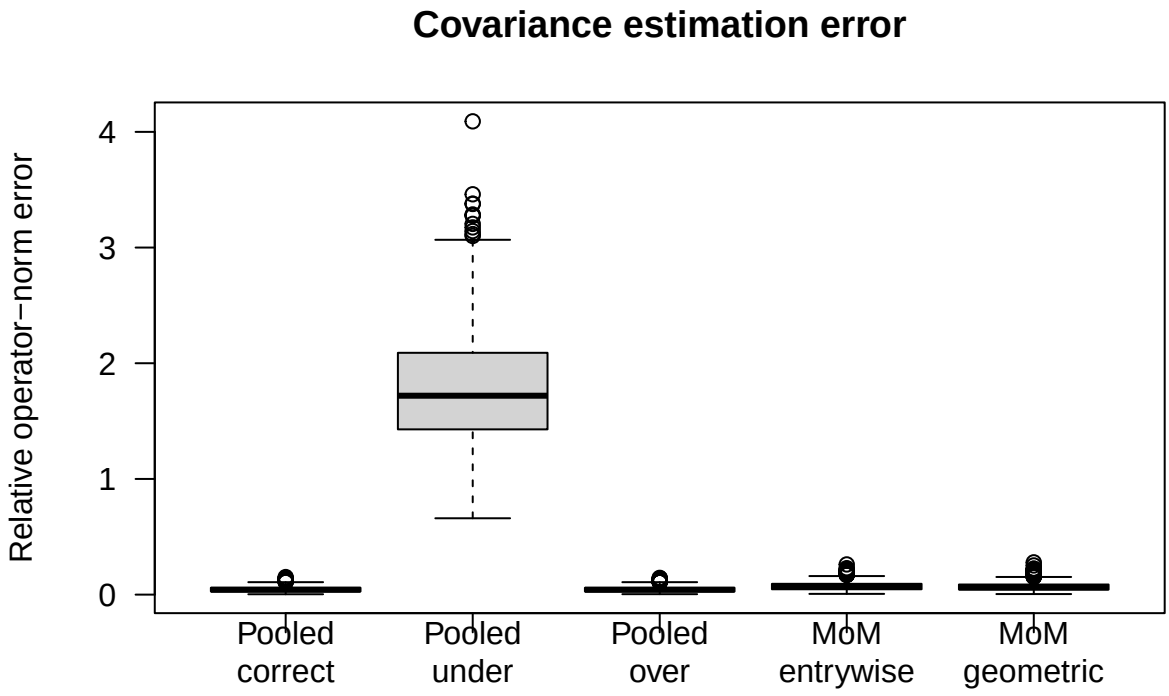


8.2 Relative operator-norm error

```
aggregate(
  relative_operator_error ~
    covariance_method + segmentation,
  data = simulation_results,
  FUN = function(x) {
    c(
      mean = mean(x),
      median = median(x),
      q90 = quantile(x, 0.90)
    )
  }
)
```

```
##    covariance_method segmentation relative_operator_error.mean
## 1             pooled      correct                    0.04543456
## 2 diff_mom_entrywise  independent                    0.07191955
## 3 diff_mom_geometric  independent                    0.06846089
## 4             pooled         over                    0.04558612
## 5             pooled        under                    1.78511606
##   relative_operator_error.median relative_operator_error.q90.90%
## 1                     0.04201014                       0.07764783
## 2                     0.06609950                       0.11853702
## 3                     0.06180822                       0.11293654
## 4                     0.04189962                       0.07810722
## 5                     1.71924298                       2.47265418
```

```
boxplot(
  relative_operator_error ~ plot_group,
  data = simulation_results,
  las = 1,
  ylab = "Relative operator-norm error",
  xlab = "",
  main = "Covariance estimation error"
)
```



9 Positive definiteness

We now examine the raw covariance estimates, before applying `make_positive_definite()`.

```
pd_summary <- aggregate(
  raw_positive_definite ~
    covariance_method + segmentation,
  data = simulation_results,
  FUN = mean
)

pd_summary$non_pd_frequency <-
  1 - pd_summary$raw_positive_definite

pd_summary
```

```
##    covariance_method segmentation raw_positive_definite non_pd_frequency
## 1             pooled      correct                     1                0
## 2 diff_mom_entrywise  independent                     1                0
## 3 diff_mom_geometric  independent                     1                0
## 4             pooled         over                     1                0
## 5             pooled        under                     1                0
```

In particular, the frequencies for the two first-difference estimators are

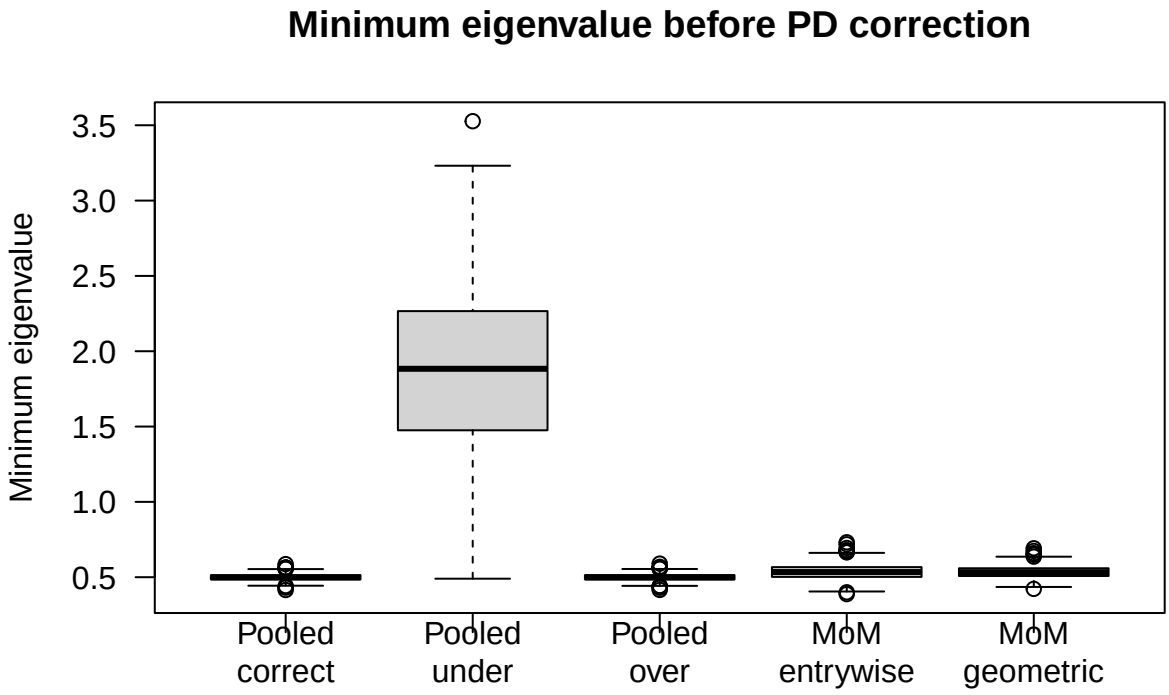
```
subset(
  pd_summary,
  covariance_method %in% c(
    "diff_mom_entrywise",
    "diff_mom_geometric"
  )
)
```

```
##    covariance_method segmentation raw_positive_definite non_pd_frequency
## 2 diff_mom_entrywise  independent                      1                0
## 3 diff_mom_geometric  independent                      1                0
```

We can also inspect the distribution of the smallest eigenvalue.

```
boxplot(
  minimum_eigenvalue ~ plot_group,
  data = simulation_results,
  las = 1,
  ylab = "Minimum eigenvalue",
  xlab = "",
  main = "Minimum eigenvalue before PD correction"
)

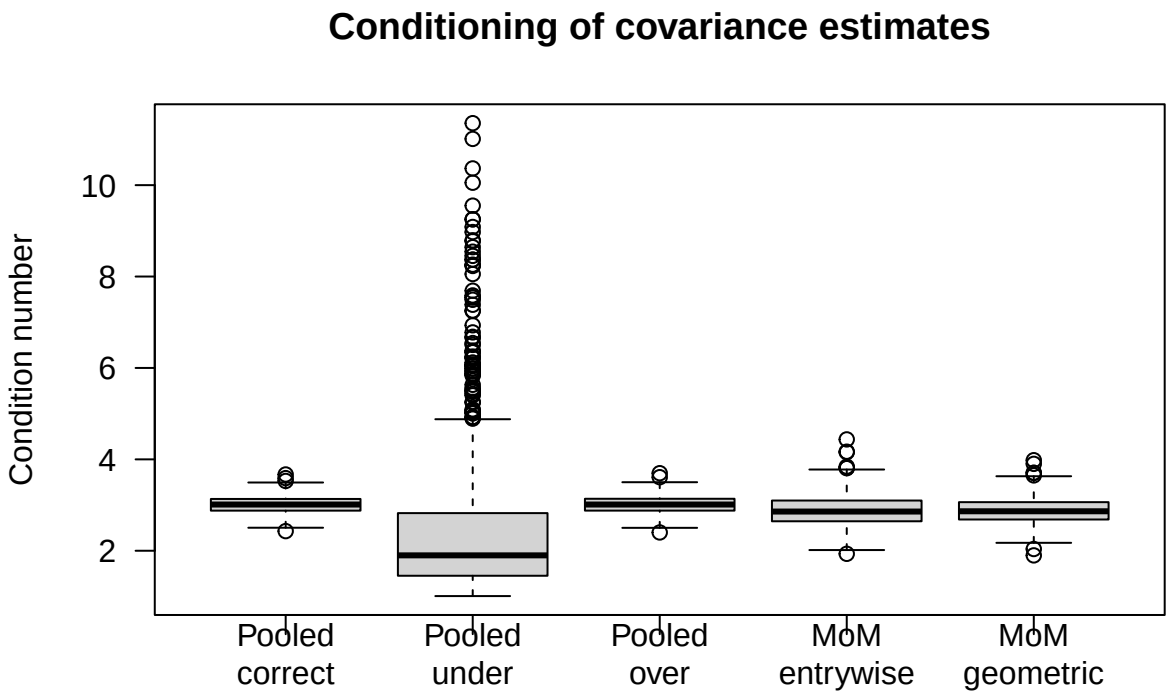
abline(
  h = 0,
  lty = 2
)
```



10 Condition number

```
boxplot(
  condition_number ~ plot_group,
  data = subset(
    simulation_results,
    is.finite(condition_number)
  ),
  las = 1,
  ylab = "Condition number",
  xlab = "",
  main = "Conditioning of covariance estimates"
```

```
)
```



11 PoSIRd coverage and confidence-region area

We now investigate how the three covariance estimators affect PoSIRd confidence regions.

For each simulated data set we consider:

- correct segmentation;
- undersegmentation;
- oversegmentation.

For each segmentation, we compute PoSIRd confidence regions using:

- pooled covariance;
- entrywise MoM covariance;
- geometric-median MoM covariance.

For an arbitrary supplied segment, the target parameter is the mean of the true signal over that segment.

12 True signal and targets

12.1 True signal at every observation

```
get_true_signal <- function(
    n,
    true_changepoints,
    true_means
) {
  boundaries <- c(
    0L,
    true_changepoints,
```

```
    n
  )

  true_signal <- matrix(
    NA_real_,
    nrow = n,
    ncol = ncol(true_means)
  )

  for (
    segment
    in seq_len(
      nrow(true_means)
    )
  ) {
    current_rows <- (
      boundaries[segment] + 1L
    ):boundaries[segment + 1L]

    true_signal[
      current_rows,
    ] <- matrix(
      true_means[segment, ],
      nrow =
        length(current_rows),
      ncol =
        ncol(true_means),
      byrow = TRUE
    )
  }

  colnames(true_signal) <-
    colnames(true_means)

  true_signal
}
```

12.2 True target associated with a supplied segmentation

```
get_true_targets <- function(
    true_signal,
    breakpoints
) {
  n <- nrow(
    true_signal
  )

  boundaries <- c(
    0L,
    breakpoints,
    n
  )
```

```
  number_of_segments <-
    length(boundaries) - 1L

  targets <- t(
    vapply(
      seq_len(
        number_of_segments
      ),
      function(segment) {
        current_rows <- (
          boundaries[segment] + 1L
        ):boundaries[segment + 1L]

        colMeans(
          true_signal[
            current_rows,
            ,
            drop = FALSE
          ]
        )
      },
      numeric(
        ncol(true_signal)
      )
    )
  )

  colnames(targets) <-
    colnames(true_signal)

  targets
}
```

13 Construct PoSIRd regions with a chosen covariance estimator

The PoSIRd quantile, estimated means, segment lengths and validity are obtained from `confidence_regions_1D()`. We then use the desired covariance estimator.

```
get_posird_regions <- function(
    x,
    breakpoints,
    covariance_method,
    delta,
    alpha,
    Qtable,
    covariance_blocks
) {
  covariance_method <- match.arg(
    covariance_method,
    c(
      "pooled",
      "diff_mom_entrywise",
      "diff_mom_geometric"
```

```
    )
  )

  regions <- confidence_regions_1D(
    x = x,
    breakpoints = breakpoints,
    delta = delta,
    alpha = alpha,
    Qtable = Qtable
  )

  covariance <- switch(
    covariance_method,

    pooled = {
      estimate_covariance_pooled(
        x = x,
        means = regions$means,
        lengths = regions$lengths
      )
    },

    diff_mom_entrywise = {
      estimate_covariance_diff_mom(
        x = x,
        number_of_blocks =
          covariance_blocks,
        aggregation =
          "entrywise"
      )
    },

    diff_mom_geometric = {
      estimate_covariance_diff_mom(
        x = x,
        number_of_blocks =
          covariance_blocks,
        aggregation =
          "geometric"
      )
    }
  )

  regions$covariance <-
    covariance

  regions
}
```

14 Evaluate coverage and ellipse area

For segment k, coverage is evaluated using

$$n_k(\widehat{\mu}_k - \mu_k)^\top \widehat{\Sigma}^{-1}(\widehat{\mu}_k - \mu_k) \leq q^2.$$

For a two-dimensional signal, the area of the corresponding confidence ellipse is

$$\pi \frac{q^2}{n_k} \sqrt{\det(\widehat{\Sigma})}.$$

```
evaluate_posird_regions <- function(
    regions,
    true_targets
) {
  number_of_segments <-
    nrow(
      regions$means
    )

  if (
    nrow(true_targets) !=
      number_of_segments
  ) {
    stop(
      paste(
        "The number of true targets",
        "does not match the number",
        "of supplied segments."
      )
    )
  }

  segment_coverage <- rep(
    NA,
    number_of_segments
  )

  areas <- rep(
    NA_real_,
    number_of_segments
  )

  critical_value <-
    regions$quantile^2

  covariance_factor <- sqrt(
    det(
      regions$covariance
    )
  )

  for (
    segment
    in seq_len(
      number_of_segments
    )
```

```
) {

  if (
    !regions$valid[segment]
  ) {
    next
  }

  distance_squared <- mahalanobis(
    x = matrix(
      true_targets[
        segment,
      ],
      nrow = 1L
    ),
    center =
      regions$means[
        segment,
      ],
    cov =
      regions$covariance
  )

  statistic <-
    regions$lengths[segment] *
    distance_squared

  segment_coverage[segment] <-
    statistic <=
    critical_value

  areas[segment] <-
    pi *
    critical_value /
    regions$lengths[segment] *
    covariance_factor
}

valid_coverage <-
  segment_coverage[
    regions$valid
  ]

valid_areas <-
  areas[
    regions$valid
  ]

list(
  segment_coverage =
    segment_coverage,

  simultaneous_coverage =
```

```
      all(valid_coverage),

    mean_area =
      mean(
        valid_areas
      ),

    median_area =
      median(
        valid_areas
      ),

    minimum_area =
      min(
        valid_areas
      ),

    maximum_area =
      max(
        valid_areas
      ),

    n_segments =
      number_of_segments,

    n_valid_segments =
      sum(
        regions$valid
      )
  )
}
```

15 One PoSIRd example

We first use the signal generated above.

```
true_signal <- get_true_signal(
  n = n,
  true_changepoints =
    true_chg,
  true_means =
    true_means
)

posird_segmentations <- list(
  correct =
    true_chg,

  under =
    undersegment(
      true_chg
    ),
```

```
  over =
    oversegment(
      true_changepoints =
        true_chg,
      n = n
    )
)
```

Inspect the number and lengths of the supplied segments.

```
lapply(
  posird_segmentations,
  function(breakpoints) {
    diff(
      c(
        0L,
        breakpoints,
        n
      )
    )
  }
)
```

```
## $correct
##  [1]  83  92  99  93  90  83  94  99 106  85  76
##
## $under
## [1] 175 192 173 193 191  76
##
## $over
##  [1] 41 42 46 46 49 50 46 47 45 45 41 42 47 47 49 50 53 53 42 43 38 38
```

Compute the true targets for each segmentation.

```
true_targets <- lapply(
  posird_segmentations,
  function(breakpoints) {
    get_true_targets(
      true_signal =
        true_signal,
      breakpoints =
        breakpoints
    )
  }
)

true_targets
```

```
## $correct
##       mu1 mu2
##  [1,]   0   2
##  [2,]  -2  -2
##  [3,]   2   2
##  [4,]  -2  -2
##  [5,]   0   2
##  [6,]   2   0
```

```
##  [7,]   0   2
##  [8,]   2   0
##  [9,]  -2   2
## [10,]   2  -2
## [11,]   0   2
##
## $under
##             mu1        mu2
## [1,] -1.0514286 -0.1028571
## [2,]  0.0625000  0.0625000
## [3,]  0.9595376  1.0404624
## [4,]  1.0259067  0.9740933
## [5,] -0.2198953  0.2198953
## [6,]  0.0000000  2.0000000
##
## $over
##       mu1 mu2
##  [1,]   0   2
##  [2,]   0   2
##  [3,]  -2  -2
##  [4,]  -2  -2
##  [5,]   2   2
##  [6,]   2   2
##  [7,]  -2  -2
##  [8,]  -2  -2
##  [9,]   0   2
## [10,]   0   2
## [11,]   2   0
## [12,]   2   0
## [13,]   0   2
## [14,]   0   2
## [15,]   2   0
## [16,]   2   0
## [17,]  -2   2
## [18,]  -2   2
## [19,]   2  -2
## [20,]   2  -2
## [21,]   0   2
## [22,]   0   2
```

Example using undersegmentation and the pooled estimator.

```
regions_example <-
  get_posird_regions(
    x = x,
    breakpoints =
      posird_segmentations$under,
    covariance_method =
      "pooled",
    delta = delta,
    alpha = alpha,
    Qtable = Qtable,
    covariance_blocks =
      covariance_blocks
  )
```

```
evaluate_posird_regions(
  regions =
    regions_example,
  true_targets =
    true_targets$under
)
```

```
## $segment_coverage
## [1] TRUE TRUE TRUE TRUE TRUE TRUE
##
## $simultaneous_coverage
## [1] TRUE
##
## $mean_area
## [1] 1.42695
##
## $median_area
## [1] 1.163788
##
## $minimum_area
## [1] 1.101379
##
## $maximum_area
## [1] 2.796924
##
## $n_segments
## [1] 6
##
## $n_valid_segments
## [1] 6
```

16 Monte Carlo evaluation of PoSIRd

For this experiment, we keep the same underlying signal across Monte Carlo replicates and generate new Gaussian noise, as in the previous PoSIRd coverage simulation.

```
oversegment <- function(
    true_changepoints,
    n,
    delta
) {
  boundaries <- c(
    0L,
    true_changepoints,
    n
  )

  minimum_length <- ceiling(
    delta * n
  )

  additional_changepoints <- integer(0)
```

```
  for (
    segment in seq_len(
      length(boundaries) - 1L
    )
  ) {
    left <- boundaries[segment]
    right <- boundaries[segment + 1L]

    segment_length <- right - left

    if (segment_length >= 2L * minimum_length) {
      midpoint <- as.integer(
        floor(
          (left + right) / 2
        )
      )

      if (
        midpoint - left >= minimum_length &&
        right - midpoint >= minimum_length
      ) {
        additional_changepoints <- c(
          additional_changepoints,
          midpoint
        )
      }
    }
  }

  sort(
    unique(
      c(
        true_changepoints,
        additional_changepoints
      )
    )
  )
}

n_sim_posird <- 1000

covariance_methods <- c(
  "pooled",
  "diff_mom_entrywise",
  "diff_mom_geometric"
)

segmentation_methods <- c(
  "correct",
  "under",
  "over"
)
```

```
true_signal <- get_true_signal(
  n = n,
  true_changepoints =
    true_chg,
  true_means =
    true_means
)

fixed_segmentations <- list(
  correct =
    true_chg,

  under =
    undersegment(
      true_chg
    ),

  over =
    oversegment(
      true_changepoints =
        true_chg,
      n = n,
      delta = delta
    )
)

fixed_targets <- lapply(
  fixed_segmentations,
  function(breakpoints) {
    get_true_targets(
      true_signal =
        true_signal,
      breakpoints =
        breakpoints
    )
  }
)
```

```
set.seed(1528)

posird_results <- vector(
  "list",
  n_sim_posird *
    length(
      covariance_methods
    ) *
    length(
      segmentation_methods
    )
)

result_index <- 1L

for (
```

```
  sim
  in seq_len(
    n_sim_posird
  )
) {

  # Generate a new data set with the same signal
  x <- generate_data(
    n = n,
    changepoints =
      true_chg,
    means =
      true_means,
    Sigma =
      Sigma
  )

  for (
    segmentation_name
    in names(
      fixed_segmentations
    )
  ) {

    current_breakpoints <-
      fixed_segmentations[[segmentation_name]]

    current_targets <-
      fixed_targets[[segmentation_name]]

    for (
      covariance_method
      in covariance_methods
    ) {

      regions <-
        get_posird_regions(
          x = x,
          breakpoints =
            current_breakpoints,
          covariance_method =
            covariance_method,
          delta = delta,
          alpha = alpha,
          Qtable = Qtable,
          covariance_blocks =
            covariance_blocks
        )

      evaluation <-
        evaluate_posird_regions(
          regions =
            regions,
```

```
          true_targets =
            current_targets
        )

      posird_results[[result_index]] <- data.frame(
        simulation =
          sim,

        segmentation =
          segmentation_name,

        covariance_method =
          covariance_method,

        simultaneous_coverage =
          evaluation$
            simultaneous_coverage,

        mean_area =
          evaluation$
            mean_area,

        median_area =
          evaluation$
            median_area,

        minimum_area =
          evaluation$
            minimum_area,

        maximum_area =
          evaluation$
            maximum_area,

        n_segments =
          evaluation$
            n_segments,

        n_valid_segments =
          evaluation$
            n_valid_segments
      )

      result_index <-
        result_index + 1L
    }
  }

  if (
    sim %% 100L == 0L
  ) {
    message(
      "PoSIRd simulation ",
```

```
      sim,
      "/",
      n_sim_posird
    )
  }
}
```

```
## PoSIRd simulation 100/1000
## PoSIRd simulation 200/1000
## PoSIRd simulation 300/1000
## PoSIRd simulation 400/1000
## PoSIRd simulation 500/1000
## PoSIRd simulation 600/1000
## PoSIRd simulation 700/1000
## PoSIRd simulation 800/1000
## PoSIRd simulation 900/1000
## PoSIRd simulation 1000/1000
```

```
posird_results <- do.call(
  rbind,
  posird_results
)

rownames(
  posird_results
) <- NULL
```

17 Simultaneous coverage

```
coverage_summary <- aggregate(
  simultaneous_coverage ~
    segmentation +
    covariance_method,
  data = posird_results,
  FUN = mean
)

coverage_summary$nominal <-
  1 - alpha

coverage_summary
```

```
##   segmentation  covariance_method simultaneous_coverage nominal
## 1      correct diff_mom_entrywise                     1    0.95
## 2         over diff_mom_entrywise                     1    0.95
## 3        under diff_mom_entrywise                     1    0.95
## 4      correct diff_mom_geometric                     1    0.95
## 5         over diff_mom_geometric                     1    0.95
## 6        under diff_mom_geometric                     1    0.95
```

```
## 7      correct             pooled                    1    0.95
## 8         over             pooled                    1    0.95
## 9        under             pooled                    1    0.95
```

18 Confidence-region areas

18.1 Numerical summary

```
area_summary_mean <- aggregate(
  mean_area ~
    segmentation +
    covariance_method,
  data = posird_results,
  FUN = function(values) {
    c(
      mean =
        mean(values),

      median =
        median(values),

      q10 =
        quantile(
          values,
          0.10
        ),

      q90 =
        quantile(
          values,
          0.90
        )
    )
  }
)

area_summary_mean
```

```
##   segmentation  covariance_method mean_area.mean mean_area.median
## 1      correct diff_mom_entrywise      0.6408156        0.6407119
## 2         over diff_mom_entrywise      0.7236545        0.7235374
## 3        under diff_mom_entrywise      0.3877803        0.3877175
## 4      correct diff_mom_geometric      0.6398627        0.6392674
## 5         over diff_mom_geometric      0.7225784        0.7219061
## 6        under diff_mom_geometric      0.3872037        0.3868434
## 7      correct             pooled      0.6078411        0.6076085
## 8         over             pooled      0.6863835        0.6862712
## 9        under             pooled      1.3980900        1.3993754
##   mean_area.q10.10% mean_area.q90.90%
## 1         0.5998017         0.6831678
## 2         0.6773387         0.7714816
## 3         0.3629613         0.4134091
## 4         0.6063488         0.6747188
```

```
## 5         0.6847322         0.7619404
## 6         0.3669232         0.4082963
## 7         0.5827970         0.6334511
## 8         0.6579794         0.7154432
## 9         1.3574457         1.4348781
```

```r
area_summary_median <- aggregate(
  median_area ~
    segmentation +
    covariance_method,
  data = posird_results,
  FUN = function(values) {
    c(
      mean =
        mean(values),

      median =
        median(values),

      q10 =
        quantile(
          values,
          0.10
        ),

      q90 =
        quantile(
          values,
          0.90
        )
    )
  }
)

area_summary_median
```

```
##   segmentation  covariance_method median_area.mean median_area.median
## 1      correct diff_mom_entrywise        0.6278896          0.6277880
## 2         over diff_mom_entrywise        0.6607204          0.6606135
## 3        under diff_mom_entrywise        0.3162648          0.3162136
## 4      correct diff_mom_geometric        0.6269560          0.6263726
## 5         over diff_mom_geometric        0.6597380          0.6591241
## 6        under diff_mom_geometric        0.3157945          0.3155007
## 7      correct             pooled        0.5955803          0.5953523
## 8         over             pooled        0.6266908          0.6265882
## 9        under             pooled        1.1402504          1.1412987
##   median_area.q10.10% median_area.q90.90%
## 1           0.5877030           0.6693875
## 2           0.6184326           0.7043882
## 3           0.2960230           0.3371671
## 4           0.5941181           0.6611090
## 5           0.6251831           0.6956768
## 6           0.2992542           0.3329972
## 7           0.5710414           0.6206736
## 8           0.6007570           0.6532232
```

```
## 9           1.1071017           1.1702539
```

19 Mean ellipse area

```
posird_results$plot_group <- with(
  posird_results,
  paste(
    segmentation,
    covariance_method,
    sep = "_"
  )
)
posird_results$plot_group <- factor(
  posird_results$plot_group,
  levels = c(
    "correct_pooled",
    "correct_diff_mom_entrywise",
    "correct_diff_mom_geometric",

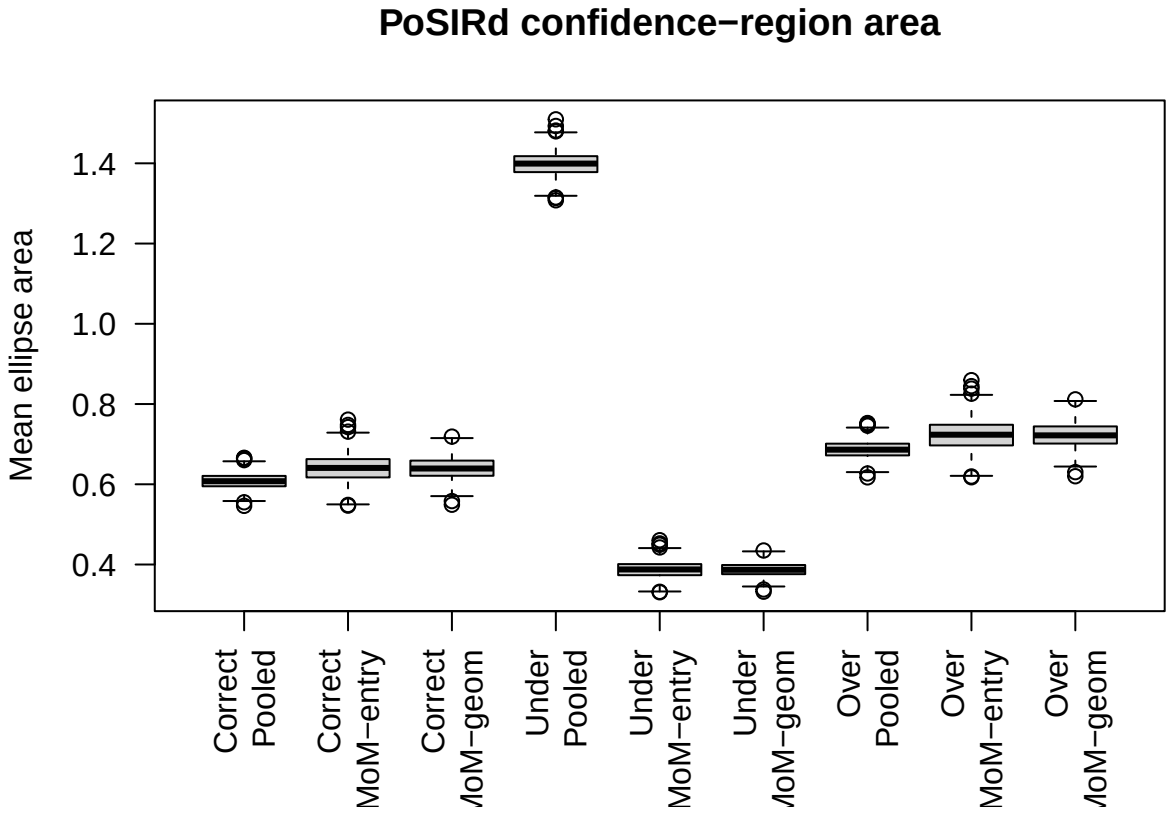
    "under_pooled",
    "under_diff_mom_entrywise",
    "under_diff_mom_geometric",

    "over_pooled",
    "over_diff_mom_entrywise",
    "over_diff_mom_geometric"
  ),
  labels = c(
    "Correct\nPooled",
    "Correct\nMoM-entry",
    "Correct\nMoM-geom",

    "Under\nPooled",
    "Under\nMoM-entry",
    "Under\nMoM-geom",

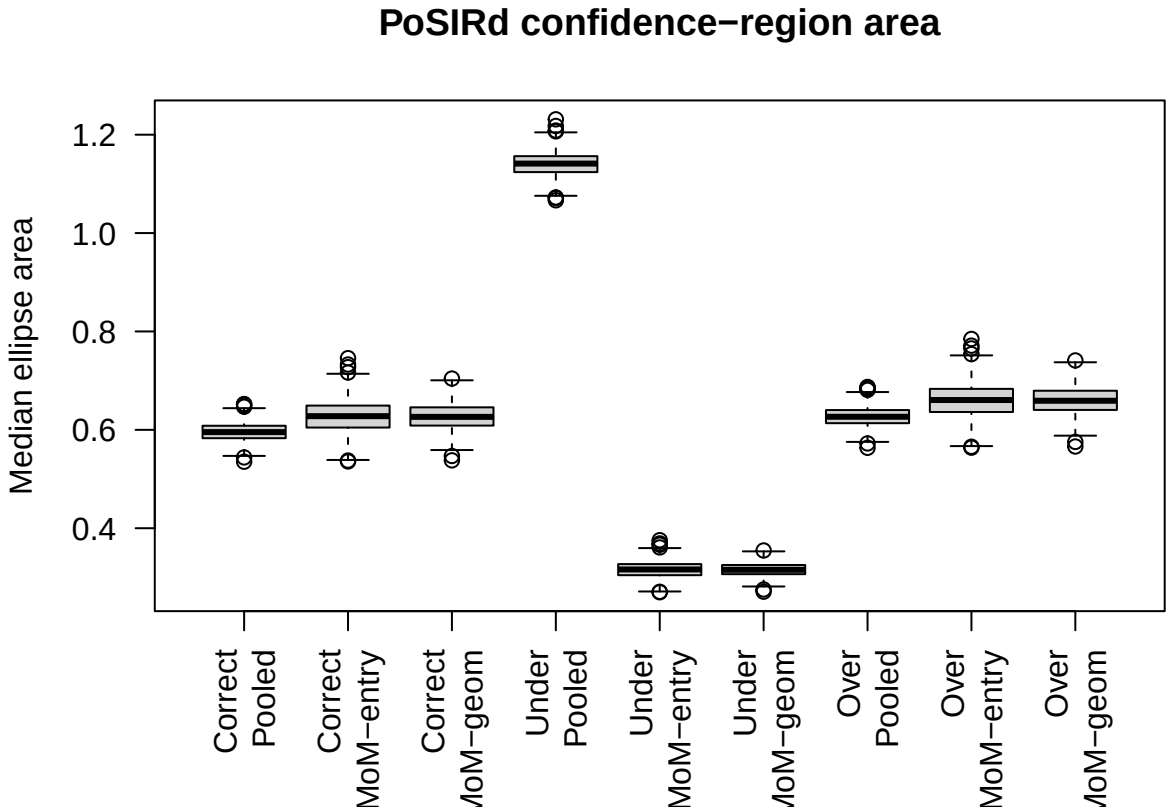
    "Over\nPooled",
    "Over\nMoM-entry",
    "Over\nMoM-geom"
  )
)

boxplot(
  mean_area ~
    plot_group,
  data = posird_results,
  las = 2,
  ylab =
    "Mean ellipse area",
  xlab = "",
  main =
    "PoSIRd confidence-region area"
)
```



20 Median ellipse area

```
boxplot(
  median_area ~
    plot_group,
  data = posird_results,
  las = 2,
  ylab =
    "Median ellipse area",
  xlab = "",
  main =
    "PoSIRd confidence-region area"
)
```



21 Relative area with respect to the correct pooled estimator

To make the magnitude of the differences easier to interpret, compare each configuration with the mean area obtained using the pooled estimator under the correct segmentation.

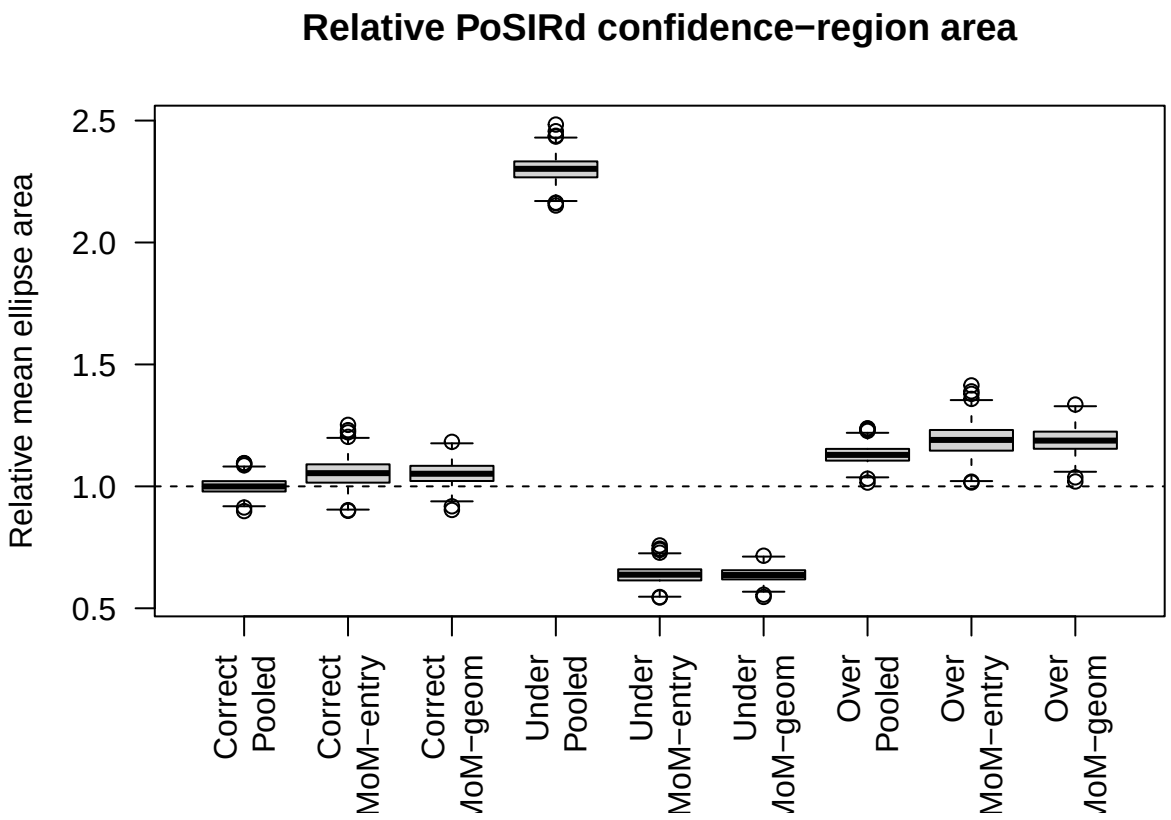
```
reference_area <- mean(
  posird_results$mean_area[
    posird_results$segmentation ==
      "correct" &
    posird_results$covariance_method ==
      "pooled"
  ]
)

posird_results$relative_area <-
  posird_results$mean_area /
  reference_area
```

```
boxplot(
  relative_area ~
    plot_group,
  data = posird_results,
  las = 2,
  ylab =
    "Relative mean ellipse area",
  xlab = "",
  main =
    "Relative PoSIRd confidence-region area"
```

```
)

abline(
  h = 1,
  lty = 2
)
```



22 Coverage and area together

```
final_summary <- merge(
  coverage_summary,
  aggregate(
    cbind(
      mean_area,
      median_area
    ) ~
      segmentation +
      covariance_method,
    data = posird_results,
    FUN = mean
  ),
  by = c(
    "segmentation",
    "covariance_method"
  )
)
```

```
final_summary
```

```
##   segmentation  covariance_method simultaneous_coverage nominal mean_area
## 1      correct diff_mom_entrywise                     1    0.95 0.6408156
## 2      correct diff_mom_geometric                     1    0.95 0.6398627
## 3      correct             pooled                     1    0.95 0.6078411
## 4         over diff_mom_entrywise                     1    0.95 0.7236545
## 5         over diff_mom_geometric                     1    0.95 0.7225784
## 6         over             pooled                     1    0.95 0.6863835
## 7        under diff_mom_entrywise                     1    0.95 0.3877803
## 8        under diff_mom_geometric                     1    0.95 0.3872037
## 9        under             pooled                     1    0.95 1.3980900
##   median_area
## 1   0.6278896
## 2   0.6269560
## 3   0.5955803
## 4   0.6607204
## 5   0.6597380
## 6   0.6266908
## 7   0.3162648
## 8   0.3157945
## 9   1.1402504
```

23 Interpretation

The most important comparison is the undersegmentation scenario.

When true changepoints are missed, the pooled residual covariance estimator contains residual variation due to changes in the signal mean. We therefore expect the pooled covariance estimate to become inflated, particularly in the directions corresponding to the missed jumps.

This inflation may not necessarily cause undercoverage. Instead, it can lead to larger PoSIRd confidence ellipses and therefore conservative simultaneous coverage.

The first-difference estimators are independent of the supplied segmentation, so their covariance estimates are unchanged across the three segmentation scenarios for a fixed data set. However, the PoSIRd regions still depend on the segmentation through the estimated segment means, lengths and target parameters.

The coverage results should therefore always be interpreted together with the ellipse areas.